Complete ML Workflow

Phases 1 – 10 · Planning → Deployment & Monitoring

Phase 1 — Planning

1	**Define Goal**	Choose task type: Classification / Regression / Clustering
1a	**Baseline Heuristic**	Majority class (classification) / Mean (regression) / Rule-based
1b	**Business Metrics**	Precision / Recall thresholds, cost matrix, revenue impact

Phase 2 — Data Validation

2	**Select Dataset**	Source: CSV / Database / API / Kaggle / custom collection
2a	**Schema Validation**	Tools: Great Expectations, Pandera
2b	**Data Contract Checks**	Null rates, value ranges, referential integrity checks
2c	**Data Contract & Schema Drift Rules ★ NEW**	Define formal data contracts: expected schema, types, value ranges, and SLAs per feature Monitor for schema drift: detect added / removed / renamed columns between pipeline runs Tools: Great Expectations checkpoints, Pandera SchemaModel versioning, Soda Core Alert on breaking changes; block downstream training until contract is re-validated

Phase 3 — Exploratory Data Analysis (raw data only)

3	**Shape & Types** .shape, .dtypes, .info()
4	**Descriptive Stats** .describe(), skew, kurtosis
5	**Missing Map** msno.matrix / sns.heatmap
6	**Distributions** Histogram, KDE, boxplot, violin plot
7	**Correlation Map** df.corr() + sns.heatmap()
8	**Target Distribution** countplot, imbalance check

Phase 4 — Data Cleaning

9	**Fix dtypes & Columns** astype(), rename(), drop useless columns
10	**Remove Duplicates** .drop_duplicates()
11	**Fix Strings** strip(), lower(), regex replace

Phase 5 — Train / Test Split

12	**Train / Test Split** train_test_split(X, y, test_size=0.2, stratify=y, random_state=42) Everything below fits on train data only
12a	**Target (y) Transformation** log1p, Box-Cox, PowerTransformer
12b	**Inverse at Predict Time** np.expm1() / inverse_transform()

Phase 6 — Preprocessing & Feature Engineering

13	**Handle Class Imbalance (train only)**	SMOTE / class_weight / scale_pos_weight — use ImbPipeline
14a	**Impute — Numeric**	SimpleImputer / KNNImputer
14b	**Impute — Categorical**	SimpleImputer(strategy='most_frequent')
15a	**Outlier Clipping — Numeric**	IQR clip / RobustScaler
15b	**Encode — Categorical**	OneHotEncoder / OrdinalEncoder
16a	**Scale — Numeric**	StandardScaler / MinMaxScaler / RobustScaler / MaxAbsScaler
16b	**Target Encode**	TargetEncoder — for high-cardinality columns only
17	**Feature Engineering**	PolynomialFeatures, FunctionTransformer — as Pipeline mlstep
18	**Feature Selection**	SelectKBest, RFE, feature importances — as Pipeline mlstep

Phase 7 — Modeling

19	**Model — Final Pipeline Step**	Pipeline([..., ('model', XGBClassifier())])
20a	**Hyperparameter Tuning**	GridSearchCV / RandomizedSearchCV / Optuna
20b	**Experiment Tracking**	MLflow / W&B; — log params, metrics, artifacts per run
21	**Cross-Validation**	StratifiedKFold + cross_val_score(pipeline, X_train, y_train)
22	**Fit Pipeline**	pipeline.fit(X_train, y_train) Scaler/encoder fitted on train → transform() on X_test pipeline.predict(X_test) — no leakage

Phase 8 — Evaluation on Test Set

23 Evaluate on Test Set
- Metrics: confusion matrix, ROC-AUC, F1, RMSE, R^2

24 Learning & Validation Curves
- Plot learning curve and validation curve
- Use for bias / variance diagnosis

24a Fairness & Bias Auditing / Data Slicing ★ NEW
- Slice test set by sensitive attributes (age, gender, race, geography) and evaluate metrics per slice
- Fairness metrics: demographic parity, equalized odds, disparate impact ratio
- Tools: Fairlearn, AI Fairness 360 (AIF360), Google What-If Tool, SliceFinder
- Flag slices where performance gap vs overall metric exceeds defined threshold (e.g. >5% F1 drop)
- Document audit results in model card before promotion to registry

Phase 9 — Explainability & Model Registry

25 Model Explainability
- SHAP values, permutation importance, LIME

25a Model Registry
- MLflow Registry / W&B; Artifacts

25b Version Control
- Staging → Production promotion workflow

25c CI/CD Automated Smoke Tests & Regression Check ★ NEW
- Run automated smoke tests on every model candidate before Staging promotion
- Regression check: new model must match or beat champion on held-out reference set
- Smoke test suite: predict on golden dataset, assert output schema, latency < threshold, no NaN outputs
- Tools: GitHub Actions / GitLab CI + pytest; integrate with MLflow Model Validation API
- Gate: block promotion to Production if any test fails (cite: 101)

Phase 10 — Deployment & Monitoring

26 Save & Deploy Pipeline
- Serialisation: joblib / ONNX
- Serving: FastAPI / Docker

26a Feature Store Integration & Security Scan ★ NEW
- Register all production features in a Feature Store for consistency between training and serving
- Feature Store options: Feast, Tecton, Hopsworks, AWS SageMaker Feature Store, Vertex AI Feature Store
- Eliminate training-serving skew: serving pipeline reads features from the same store used at train time
- Security scan: run SAST / dependency audit on model artefact and API container before release
- Scan tools: Bandit (Python), Trivy (container CVE scan), OWASP Dependency-Check (cite: 107)
- Store scan report alongside model card in registry; block deploy on critical CVEs

27 Monitor & Retrain
- Track: data drift, concept drift
- Scheduled retrain loop

Notes

1a / 1b	Baseline heuristic: majority class / mean + business metrics (cost matrix, precision/recall thresholds).
2a / 2b	Data validation: Great Expectations / Pandera + null rates, value ranges, referential integrity checks.
2c	Data contract & schema drift: version your contracts alongside code; treat a breaking schema change as a pipeline failure, not a warning.
12a / 12b	Target (y) transformation: log1p / Box-Cox; apply inverse transform at predict time.
20b	Experiment tracking: MLflow / W&B; logs params, metrics, and artifacts per run.
24a	Fairness audit: document slice results in a model card. A model that passes overall metrics but fails on a demographic slice should not be promoted.
25a / 25b	Model registry: version control; Staging → Production promotion before deploy.
25c	CI/CD smoke tests (cite 101): automate the champion/challenger regression check in your CI pipeline — never promote manually without a passing test suite.
26a	Feature store & security scan (cite 107): training-serving skew is one of the most common silent production failures; a feature store is the primary mitigation.

[!] Leakage Rule: Never .fit() on test data. Pipeline + ImbPipeline enforce this automatically.